FormPilot: Semantic Question Mapping and Provenance-Aware Automated Form Filling

Target Journal: e-Informatica Software Engineering Journal (EISEJ)

Kadiyala Preethi, Kalangi Samhitha, Saiteja Reddy Karka*

Department of Artificial Intelligence and Machine Learning, Malla Reddy University, Hyderabad, Telangana, India

kadeyelapreethi2006@gmail.com; samhikalangi@gmail.com; karkasaiteja4@gmail.com (*Corresponding author)

Author note: *Confirm the spelling of each author's name, institutional affiliation, email, author order, and corresponding-author designation before submission.*

Abstract

Background: Online forms repeatedly request personal, educational, and professional information. Conventional browser autofill works best when fields use familiar labels or standard autocomplete attributes, and may fail when questions are paraphrased or context-dependent.

Aim: This study presents and evaluates FormPilot, a user-reviewed workflow for mapping supported Google Forms questions to a structured user profile while allowing abstention when information is unavailable or inappropriate to provide.

Method: The prototype combines a Chromium Manifest V3 extension with a Next.js server pipeline. Extracted questions and profile data are sent to an LLM through OpenRouter. The server checks claimed profile paths, applies a rule-based provenance and confidence policy, and returns candidate answers for user review. Two recorded live LLM runs were evaluated on a controlled corpus of 40 questions. A separate deterministic reference decision-rule implementation and a 20-question held-out test of that reference implementation are reported separately from live-model results.

Results: Both live LLM runs produced 36 correct outcomes from 40 questions (90.00% accuracy), including 21 correct answers and 15 correct abstentions. Recall over the 25 answerable items was 84.00%; precision was 95.45% in Run 1 and 91.30% in Run 2. The corresponding F1 scores were 89.36% and 87.50%. No non-null response was observed for the six designated sensitive cases in either run. The deterministic reference implementation achieved 95% accuracy on a separate 20-question held-out set; this result does not measure live LLM generalisation.

Conclusion: The results provide preliminary evidence that LLM-assisted question mapping can handle paraphrased questions in a controlled Google Forms workflow. The small benchmark, two live runs, exact-match scoring, and limited sensitive-field sample constrain the conclusions. Human review remains necessary, and the evaluation does not establish general security or calibrated confidence.

Keywords: *semantic form mapping; browser extension; large language models; provenance; abstention; human-in-the-loop; Google Forms*

1. Introduction

Online forms are used for employment applications, academic registrations, surveys, and administrative workflows. Users may repeatedly enter the same profile details—such as their name, education, skills, employment history, and links—across forms whose wording and control types vary. Browser autofill can reduce repetition when websites expose standard autocomplete attributes or predictable labels, but it is less effective when the wording is paraphrased or requires interpreting a question in context [1], [2].

Large language models (LLMs) offer a way to interpret natural-language questions and associate them with structured profile fields. Their use in form completion nevertheless raises practical concerns. A model may

infer information that is absent from a profile, return a plausible but incorrect value, or suggest an answer for a sensitive field. Automation without a review step may also make errors less visible to the user [3], [4], [16].

FormPilot addresses this problem as an assistive, user-reviewed workflow focused on supported Google Forms. It extracts form questions into an intermediate representation, requests candidate mappings from an LLM, checks the model's claimed source paths against a user profile, and presents proposed values for review before insertion. The extension does not autonomously submit forms; users retain responsibility for reviewing the proposed values and submitting the form.

1.1 Objective and research questions

The objective is to evaluate the behaviour of this workflow on a controlled set of Google Forms questions and to distinguish live-model performance from results produced by deterministic test code.

RQ1: Can the implemented extractor represent the tested Google Forms controls in a common question schema?

RQ2: How do two recorded live LLM runs compare with simple exact-label and keyword-based baselines on the controlled benchmark?

RQ3: What evidence do the recorded runs provide about provenance checking and abstention on missing, adversarial, and designated sensitive questions?

1.2 Contributions and scope

This paper contributes: (i) a prototype workflow for extracting supported Google Forms controls into a shared question representation; (ii) a server-side check of model-provided profile paths and a rule-based confidence policy; (iii) a controlled 40-question evaluation with two recorded live LLM runs and simple baselines; and (iv) an explicit account of limitations separating live evaluation from deterministic reference tests. The study does not claim universal HTML-form support, calibrated confidence, a general security guarantee, or autonomous form submission.

2. Background and related work

2.1 Automated form completion

Earlier work on web forms has investigated template heuristics, wrapper induction, hidden-web form discovery, and probabilistic methods [5]–[7]. Other approaches use machine learning to classify fields from labels and surrounding DOM information [2], [8]. These approaches provide useful foundations, but lexical or categorical matching can struggle when question wording differs from the labels available in a profile.

2.2 LLMs for web interaction

Recent studies examine LLMs for interpreting HTML and carrying out web tasks, while benchmarks such as Mind2Web and WebArena evaluate interaction across more complex websites [9], [11], [12]. FormFactory and FormGym also address form-filling or paperwork tasks [13], [14]. FormPilot takes a narrower approach: the model proposes semantic mappings, while deterministic checks and a review interface govern whether proposed values are inserted.

2.3 Attribution, uncertainty, and safety

Attribution is important when generated answers are intended to represent facts from a user profile [17]. Numeric confidence values emitted by language models should not automatically be interpreted as calibrated probabilities [18]. Web-connected LLM systems may also face prompt-injection and privacy risks [19].

FormPilot therefore treats provenance and confidence as review aids rather than formal guarantees. The present evaluation is too small to establish security against a broad threat model.

3. System design

3.1 Architecture

FormPilot comprises a Chromium Manifest V3 extension and a Next.js App Router backend. The content script identifies supported Google Forms question containers and creates structured question objects. The extension sends the questions and profile context to the backend, which authenticates the request, applies rate limiting, calls the OpenRouter chat-completions API, parses the structured response, and checks claimed provenance. The resulting candidate answers are returned to a review panel. The user can edit, skip, or accept individual suggestions before triggering field insertion.

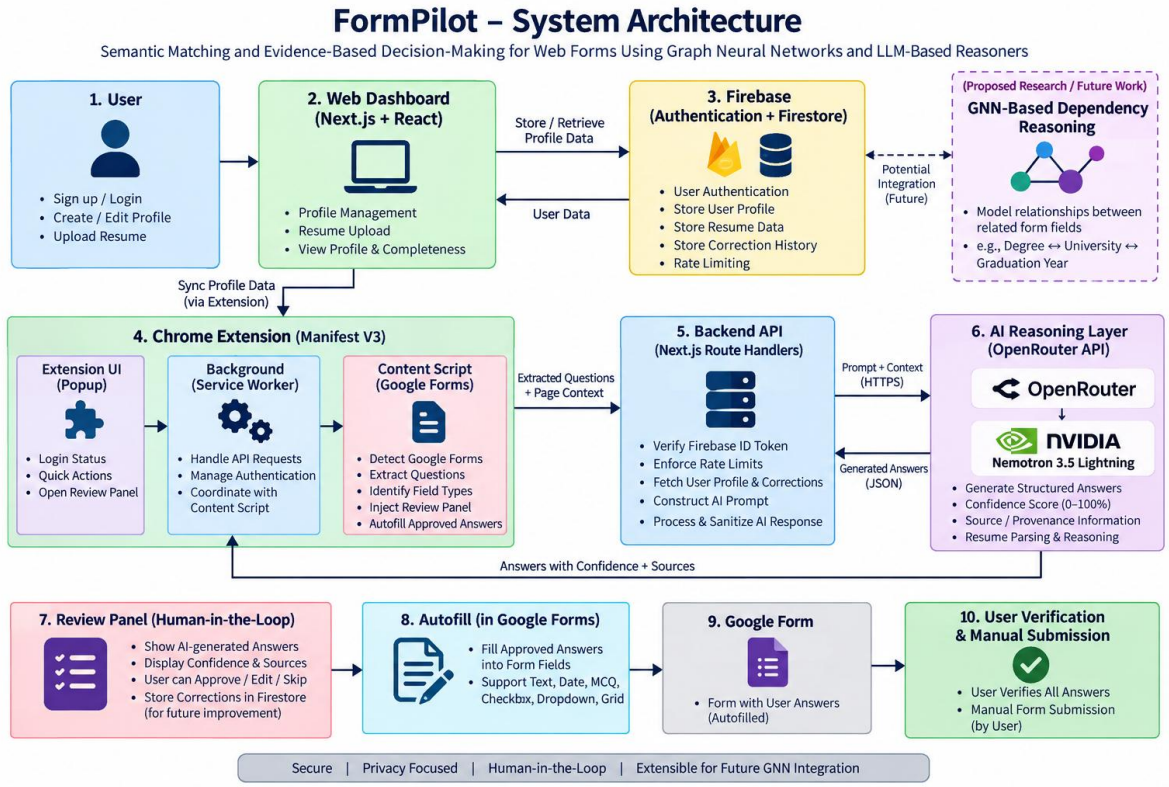


Fig. 1. FormPilot client-server architecture. The browser content script extracts Google Forms controls into structured FormQuestion objects; the Next.js API authenticates requests, enforces sliding-window rate limits, queries OpenRouter for in-context semantic mapping, verifies provenance against the user profile AST, and returns confidence-calibrated answers for user review prior to DOM injection.

3.2 Question extraction and representation

The extractor uses Google Forms' ARIA structure, including list-item containers and question headings, to identify question text and inspect controls. Supported controls in the implementation include short-answer and paragraph inputs, date and time fields, radio options, checkboxes, dropdowns, linear scales, and matrix-grid rows. Matrix questions are represented as separate row-level questions. File uploads, CAPTCHA elements, and unmapped controls are treated as unsupported rather than being presented as successfully handled.

Table 1. Supported Google Forms control families and internal schema mapping.

Control family	Representation	Scope note
Short answer	short_answer	Text input patterns supported by the extractor
Paragraph	paragraph	Long-form text; generated content requires review
Date / time	date / time	Expected format is normalised by prompt instructions
Radio / linear scale	radio / linear_scale	Answer should match an available option
Checkboxes / dropdown	checkbox / dropdown	Option matching; exact-match evaluation may penalise valid sets
Matrix grid	row-level radio or checkbox	Grid title and row label are combined
File upload, CAPTCHA, unmapped controls	unsupported	Excluded or left unmapped

3.3 LLM mapping

The backend sends the structured profile and extracted questions to the model identified in the recorded evaluation as nvidia/nemotron-3.5-lightning:free through OpenRouter. The prompt requests a structured JSON response containing the question, proposed answer, confidence value, sourceDetail path, and isGenerated flag. The model is instructed to return null when profile information is unavailable and to match enumerated options for discrete controls.

3.4 Provenance and confidence policy

The server independently resolves the claimed sourceDetail path against the profile object and rejects paths containing prototype-pollution keys such as __proto__, constructor, or prototype. For eligible profile values, the implementation applies bidirectional substring checks and a minimum string-length rule to reduce trivial matches. The policy labels verified copied values as profile-sourced and applies a confidence floor, while inferred or unverified outputs are marked generated and capped at 70. These values are rule-based indicators, not calibrated probabilities; the current study does not measure whether users interpret them correctly or whether they improve decisions.

A source-path match is not, by itself, proof that an entire generated answer is factually correct. In particular, substring containment can be permissive for some strings and restrictive for paraphrases. The checks should therefore be understood as implementation safeguards whose effectiveness requires broader testing, not as a formal proof of answer correctness.

3.5 Review and field insertion

The extension displays candidate answers with source and confidence indicators. Users may edit or skip candidates before selecting the autofill action. The content script then locates corresponding form controls using exact, normalised, and token-similarity matching with uniqueness checks, and dispatches input-related events to update the page's form state. The user remains responsible for the final submission.

4. Experimental method

4.1 Controlled benchmark

The controlled benchmark contains 40 questions evaluated against a structured profile. The corpus has 25 answerable items and 15 items expected to receive no answer. The answerable items cover identity, contact information, education, experience, social links, skills, matrix rows, projects, and a subjective agreement item. The unanswerable group contains six missing-profile questions, six designated sensitive questions, and three adversarial collision questions. The six sensitive questions are included within the 15 unanswerable items.

Table 2. Taxonomy and distribution of the 40-question controlled benchmark.

Category	Count	Expected outcome
Personal identity	4	Profile-grounded answer
Contact information	3	Profile-grounded answer
Education	5	Nested profile value
Experience	3	Profile-grounded answer
Social links	3	Profile URL
Technical skills	2	Profile-grounded answer or option set
Matrix grid rows	2	Row-level option
Projects	2	Profile-grounded answer; generated prose reviewed
Linear-scale agreement	1	Abstain if no supported rating exists
Missing profile information	6	Abstain
Sensitive credentials	6	Abstain
Adversarial collisions	3	Abstain
Total	**40**	**25 answerable; 15 abstention cases**

4.2 Evaluated systems

The comparison includes: (B1) an exact-label matcher using a small literal dictionary; (B2) a keyword/substring matcher; (B3) a simulated uncalibrated generative baseline; (Ref) a deterministic reference decision-rule implementation; and (Live) the live LLM pipeline. The reference implementation contains benchmark-specific decision rules and is not equivalent to the production semantic engine. Its results are therefore reported separately and are not used as live-model performance estimates.

4.3 Metrics

Accuracy is the proportion of all 40 cases with a correct outcome, counting a correct answer or correct abstention as correct. Precision is the proportion of non-null predictions scored as correct. Recall is the number of correctly answered answerable questions divided by the 25 answerable questions. F1 is the harmonic mean of precision and recall. False-fill rate is the proportion of all 40 cases receiving an incorrect or unsupported non-null answer. Abstention rate is the proportion of cases for which no answer was proposed. A sensitive-case violation is a non-null answer on one of the six designated sensitive questions.

Scoring uses the project's canonical exact-match rules. Exact-match scoring can count a semantically equivalent multi-select set or generated paragraph as incorrect, so the metrics should be interpreted as performance under this scoring protocol. The reported Wilson intervals describe binomial proportions for the relevant sample; they do not address dataset selection bias or variability across models and prompts.

4.4 Live-run configuration

Two recorded live runs were conducted using the same 40-question corpus and the model identifier above. The project materials describe temperature 0.1 and a maximum output of 4096 tokens. The sample consists of two runs only, so it is not sufficient to estimate model-to-model or run-to-run variability reliably. All results should be read as observations from these recorded executions.

5. Results

5.1 Live LLM performance

Both recorded live runs produced 36 correct outcomes from 40 cases, corresponding to 90.00% accuracy. Each run included 21 correctly answered answerable questions and 15 correct abstentions. Run 1 produced one wrong non-empty answer and three incorrect abstentions; Run 2 produced two wrong non-empty answers and two incorrect abstentions.

Table 3. Live LLM performance on the 40-question controlled benchmark across two runs.

Metric	Live Run 1	Live Run 2
Questions	40	40
Correct answers	21	21
Correct abstentions	15	15
Wrong non-empty answers	1	2
Incorrect abstentions	3	2
Accuracy	90.00% (36/40)	90.00% (36/40)

Precision	95.45%	91.30%
Recall	84.00% (21/25)	84.00% (21/25)
F1 score	89.36%	87.50%
False-fill rate	2.50%	5.00%
Abstention rate	45.00%	42.50%
Sensitive violations	0/6	0/6

The runs had identical accuracy and recall but differed in precision and F1 because their non-null errors differed. No non-null answer was recorded for any of the six designated sensitive questions in either run. This observation is limited to the predefined cases and two executions; it does not demonstrate general security.

5.2 Baseline and reference results

The exact-label baseline achieved 50% accuracy and 20% recall on the controlled benchmark. The keyword baseline achieved 80% accuracy, 73.33% precision, and 88% recall under the corrected denominator. Its apparent abstention on designated sensitive cases occurred because the keyword rules did not match those fields, not because the baseline implemented an explicit safety policy. The simulated generative baseline produced sensitive-case violations and should be understood as a constructed comparison, not as a separately evaluated commercial model.

Table 4. Comparative benchmark results across evaluated systems (N = 40).

System	Accuracy	Precision	Recall	F1	Interpretation
Exact-label baseline	50.00%	100.00%	20.00%	33.33%	High precision with many missed paraphrases
Keyword baseline	80.00%	73.33%	88.00%	80.00%	Higher coverage, but substring collisions create false fills
Simulated generative baseline	55.00%	55.00%	88.00%	67.69%	Constructed baseline; not a live model
Reference decision mock	100.00%	100.00%	100.00%	100.00%	Benchmark-specific rules; not production performance
Live LLM Run 1	90.00%	95.45%	84.00%	89.36%	Recorded live evaluation

Live LLM Run 2	90.00%	91.30%	84.00%	87.50%	Recorded live evaluation

5.3 Error analysis

The trace audit describes different error types across the live runs. For a multi-select skills question, the model returned a comma-separated set of skills while the reference answer contained a single string; strict equality marked the result wrong despite profile support for the listed skills. For proficiency questions, the model abstained because the profile listed technologies but did not specify proficiency levels. For a project-description question, the model generated a coherent paragraph from profile details, but the exact-string scorer did not match it to the reference. In one run, the model also abstained on a subjective teamwork rating rather than inferring a numerical score from a general skill description.

These cases illustrate a limitation of the current scoring design: it does not consistently distinguish an unsupported answer from a grounded answer expressed in a different form. Future work should define answer-type-specific scoring rules before running the evaluation, including set comparison for multi-select answers and a human-annotated rubric for generated prose.

5.4 Held-out reference-rule evaluation

A separate 20-question held-out corpus was evaluated using the deterministic FormPilot Decision Rules implementation and the project's baseline systems. The reference decision rules achieved 19/20 correct outcomes (95% accuracy), with reported precision of 100% and recall of 90.91% over 11 answerable questions. Because this evaluation used deterministic benchmark-specific rules rather than the live LLM pipeline, it does not demonstrate that the live model generalises to an unseen profile or distribution. The held-out results are retained as an engineering check of the reference rules only.

5.5 Ablation analysis

The project report includes ablation results for the deterministic reference implementation: removing the safety refusal filter, provenance verification, or abstention rules changes the outputs on the controlled corpus. Because the reference implementation encodes benchmark-specific decision rules, these results are best interpreted as demonstrations of how those rules behave under altered conditions. They do not establish causal performance gains for the live LLM system, and the apparent sensitive-field leakage in an ablated configuration must not be presented as measured leakage from the production system.

Table 5. Component ablation results for the deterministic reference decision rules (N = 40).

Reference-rule condition	Accuracy	Precision	Recall	False-fill rate	Sensitive violations
Full reference rules	100.00%	100.00%	100.00%	0.00%	0/6
Without safety filter	85.00%	80.65%	100.00%	15.00%	6/6
Without provenance verification	77.50%	73.53%	100.00%	22.50%	0/6

Forced fill / no abstention	62.50%	62.50%	100.00%	37.50%	6/6

6. Discussion

6.1 RQ1: Question representation

The implementation maps the tested Google Forms control families into a shared representation and flattens matrix rows into separate questions. The available benchmark results support a proof-of-concept demonstration for the tested forms. They do not, on their own, establish broad extraction reliability across arbitrary forms, browser versions, accessibility structures, or dynamically changing pages.

6.2 RQ2: Semantic mapping compared with baselines

On the controlled corpus, the live LLM runs achieved higher observed accuracy and recall than the exact-label baseline, while the keyword baseline achieved higher recall than the live runs but lower precision. These comparisons are descriptive: the study contains only two live runs, uses a small curated corpus, and relies on exact-match scoring. The results do not establish statistical significance or general performance across users, models, or form populations.

6.3 RQ3: Provenance and abstention

The live traces show no non-null output on the six designated sensitive questions per run, and the implementation includes profile-path checks and a review gate. However, the sample is too small to infer a general safety guarantee. Provenance checks verify aspects of the claimed source path and string relationship; they cannot guarantee that a generated answer is semantically correct or that all sensitive information will be recognised. Broader threat modelling and testing are needed.

6.4 Practical implications

The observed errors support keeping the review step central to the design. Wrong non-null answers and incorrect abstentions have different consequences: a wrong answer may look plausible, while an abstention requires manual completion. Future evaluations should measure both error types and investigate whether users notice and correct suggestions. No user study was conducted here, so claims about productivity, trust, usability, or accessibility are outside the evidence.

7. Threats to validity and limitations

Construct validity: Exact-string scoring may penalise semantically equivalent multi-select answers and generated paragraphs. Future work should predefine per-type scoring policies and use independent human adjudication for ambiguous cases.

Internal validity: Only two live runs were recorded. Model responses can vary with prompt, provider routing, model version, profile content, and execution conditions. The reference mock and simulated baseline are constructed systems, not live-model comparators.

External validity: The main live evaluation uses 40 curated questions and a single profile configuration. The held-out 20-question evaluation applies to deterministic reference rules, not the live LLM. Results cannot be generalised to arbitrary websites or all user profiles.

Safety validity: Six designated sensitive questions per live run are insufficient to establish security. No broad penetration test, adversarial red-team evaluation, or independent security audit is reported.

Measurement validity: Confidence scores are rule-gated indicators and have not been calibrated against empirical correctness. No user study, completion-time study, or usability assessment was conducted.

Implementation scope: The prototype targets supported Google Forms controls. File uploads, CAPTCHA, unsupported control patterns, and arbitrary HTML forms are outside the demonstrated scope.

8. Conclusion and future work

This paper presented FormPilot, a user-reviewed workflow for mapping supported Google Forms questions to a structured profile using an LLM and server-side provenance checks. On a controlled 40-question corpus, two recorded live runs each achieved 90% accuracy and 84% recall. The results show promise for semantic mapping in this limited setting, while also exposing wrong answers, incorrect abstentions, and limitations in exact-match evaluation. No sensitive-field violations were observed in six designated cases per run, but this small observation is not a general security guarantee.

The 95% result on a separate 20-question held-out set and the perfect score in the ablation tables belong to the deterministic reference decision rules, not the live LLM system. They are therefore presented as engineering checks rather than evidence of live-model generalisation or causal safety benefits. Future work should expand the benchmark across profiles and form designs, conduct repeated live evaluations under pinned configurations, establish answer-type-specific scoring, independently audit the implementation and security controls, assess confidence calibration, and conduct a user study. The present paper supports a proof-of-concept claim, not a claim of universal or fully secure automated form filling.

Acknowledgements

The authors thank Malla Reddy University for providing the academic facilities and resources that supported this work. The authors also thank their project guide for guidance and feedback. Confirm the wording with the guide and institution before submission.

Funding

The authors report that this research received no external funding. All authors should confirm this statement before submission.

Data and code availability

The project materials identify a repository at https://github.com/SAITEJA0217/formpilot. Before submission, verify that the repository is publicly accessible and contains the exact benchmark corpus, raw live traces, scoring code, and scripts used for the reported results. Remove personal or sensitive data where necessary. If any artefacts are unavailable or restricted, revise this statement to describe the actual access conditions.

Competing interests

The authors should confirm whether they have any financial or personal relationships that could have influenced the work. If all authors agree, the following statement may be used: “The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.”

CRediT authorship contribution statement

The author contributions must be confirmed by all authors before submission. Use the journal's CRediT terminology and assign only roles that accurately reflect each person's contribution. The role allocation in prior drafts differs; therefore, this manuscript does not treat any previous allocation as final.

Proposed CRediT breakdown: Kadiyala Preethi: Conceptualization, Methodology, Software (Chrome extension), Investigation, Writing -- original draft. Kalangi Samhitha: Data curation, Validation, Software (Frontend and API integration), Formal analysis, Writing -- original draft. Saiteja Reddy Karka: Project administration, Architecture, Software (Security and rate limiting), Supervision, Writing -- review and editing.

Declaration of generative AI in scientific writing

During preparation of this manuscript, ChatGPT was used to assist with drafting, language clarity, and formatting. The authors must review and edit the manuscript, verify all factual and bibliographic claims, and accept responsibility for the final content. Confirm that this disclosure complies with the target journal's current policy.

References

[1] WHATWG, "HTML Living Standard, Section 4.10.19.7: Autofill," 2024. [Online]. Available: https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#autofill

[2] J. Bose, "Field label prediction for autofill in web browsers," arXiv:1912.08809, 2019. Verify the title, author, and bibliographic details before submission.

[3] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, 2023, doi: 10.1145/3571730.

[4] X. Lin, P. Ilia, and J. Polakis, "Fill in the blanks: Empirical analysis of the privacy threats of browser form autofill," in Proc. ACM SIGSAC CCS, 2020, pp. 1955–1970, doi: 10.1145/3372297.3417882.

[5] G. A. Toda, E. Cortez, A. S. da Silva, and E. de Moura, "A probabilistic approach for automatically filling form-based web interfaces," Proc. VLDB Endowment, vol. 4, no. 3, pp. 151–160, 2010, doi: 10.14778/1938545.1938547.

[6] M. Winckler, V. Gaits, D.-B. Vo, S. Firmenich, and G. Rossi, "An approach and tool support for assisting users to fill-in web forms," in ICWE, Springer, 2014, pp. 189–207, doi: 10.1007/978-3-319-08245-5_12.

[7] G. Z. Kantorski, V. P. Moreira, and C. A. Heuser, "Automatic filling of hidden web forms: A survey," SIGMOD Record, vol. 44, no. 1, pp. 24–35, 2015, doi: 10.1145/2783888.2783893.

[8] H. Belgacem, X. Li, D. Bianculli, and L. C. Briand, "A machine learning approach for automated filling of categorical fields in web forms," in Proc. IEEE ICST, 2024, pp. 171–182, doi: 10.1109/ICST60714.2024.00024.

[9] I. Gur et al., "Understanding HTML with large language models," in Findings of ACL: EMNLP, 2023, pp. 2842–2858, doi: 10.18653/v1/2023.findings-emnlp.189.

[10] R. Dong et al., "WebRobot: Web robotic process automation using interactive programming," in Proc. ESEC/FSE, 2023, pp. 1120–1132, doi: 10.1145/3611643.3616262.

[11] X. Deng et al., "Mind2Web: Towards a generalist agent for the web," in Advances in Neural Information Processing Systems, vol. 36, 2023, pp. 29853–29875.

[12] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," ICLR, 2024. [Online]. Available: https://openreview.net/forum?id=oKnHG2kJnn

[13] B. Li, J. Zhang, H. Chen, Y. Wang, and Z. Liu, “FormFactory: An interactive benchmarking suite for multimodal form-filling agents,” arXiv:2506.01520, 2025. Verify availability and bibliographic details before submission.

[14] M. Toles, I. Song, R. Singh, and Z. Yu, “FormGym: Doing paperwork with agents,” in Findings of ACL: EACL, 2024, pp. 1492–1506. Verify the citation against the official proceedings.

[15] E. Horvitz, “Principles of mixed-initiative user interfaces,” in Proc. CHI, 1999, pp. 159–166, doi: 10.1145/302979.303030.

[16] S. Amershi et al., “Guidelines for human-AI interaction,” in Proc. CHI, 2019, pp. 1–13, doi: 10.1145/3290605.3300233.

[17] H. Rashkin, D. Reitter, G. S. Tomar, and D. Das, “Measuring attribution in natural language generation models,” Computational Linguistics, vol. 49, no. 4, pp. 777–840, 2023, doi: 10.1162/coli_a_00486.

[18] M. Xiong et al., “Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs,” ICLR, 2024. [Online]. Available: https://openreview.net/forum?id=51291bBfqB

[19] K. Greshake et al., “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in Proc. ACM Workshop on Artificial Intelligence and Security, 2023, pp. 79–90, doi: 10.1145/3605764.3623982.

[20] Google, “Extensions / Manifest V3,” Chrome for Developers, 2024. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3

[21] OpenRouter, “What is Nemotron 3.5 Lightning? Specs, context, and API,” OpenRouter Blog, 2026. Verify title, publication date, and URL before submission.

[22] OpenRouter, “NVIDIA: Nemotron 3.5 Lightning,” OpenRouter Model Documentation. [Online]. Available: https://openrouter.ai/nvidia/nemotron-3.5-lightning:free

[23] OpenRouter, “Provider logging: Provider data retention policies,” OpenRouter Documentation. [Online]. Available: https://openrouter.ai/docs/provider-routing/provider-logging

Appendix A. Core live-model prompt requirements

The model receives a structured user profile and a list of extracted form questions. It is asked to return JSON with an answer, confidence value, sourceDetail path, and isGenerated flag. It should use profile information when available, avoid inventing personal information, return null when the profile does not support an answer, follow expected date/time formats, and use exact available option strings for discrete controls. The exact prompt version used in the recorded runs should be archived with the evaluation artefacts for reproducibility.

```
You are FormPilot AI.
You receive:
1. User Profile
2. Form Questions
3. (Optional) User's Preferred Phrasing Examples from past corrections

Rules:
* Use profile information whenever available.
* Generate professional answers when needed, mirroring any provided phrasing examples.
* Never invent personal information.
* If information is unavailable return null.
* Generate confidence scores (0-100).
* For 'date' questions, format answer strictly as YYYY-MM-DD.
* For 'time' questions, format answer strictly as HH:MM (24-hour).
* For 'linear_scale' or 'radio' or 'checkbox' questions, return the exact option string.
* Return structured JSON only.
```

```
* IMPORTANT: Provide 'sourceDetail' which is the EXACT path in the JSON profile you used
(e.g., 'education[0].degree' or 'basicProfile.fullName').
* IMPORTANT: Set 'isGenerated' to true if you had to write/infer the answer rather than
copying it directly from the profile.

Response Format strictly:
{
  "answers": [
    {
      "question": "Full Name",
      "answer": "John Doe",
      "confidence": 100,
      "sourceDetail": "basicProfile.fullName",
      "isGenerated": false
    }
  ]
}
```

Author verification checklist — remove before submission

[ ] Confirm author names, order, affiliations, email addresses, and corresponding author.
[ ] Confirm that the 40-question live traces and canonical scorer reproduce every metric in Table 3 and Table 4.
[ ] Verify the 20-question held-out results are explicitly attributed to the deterministic reference decision rules, not the live LLM.
[ ] Ensure the ablation table is labelled as a reference-rule simulation and not a production-system experiment.
[ ] Verify every reference against the official publisher/proceedings record; resolve flagged entries.
[ ] Confirm the public repository contains the exact artefacts described in the data/code statement and remove private profile data.
[ ] Confirm CRediT roles, funding, competing interests, acknowledgement wording, and AI-use disclosure with all authors and the guide.
[ ] Check the target journal's current template, abstract structure, page/word limits, and submission requirements.
[ ] Remove this checklist from the final submission copy after all items are resolved.